

Relatório Técnico

Practical Assignment

Trabalho Prático

Processamento de Linguagens



Universidade do Minho

Universidade do Minho

Escola de Engenharia, Braga, Portugal
Licenciatura em Engenharia Informática

Ano Letivo 2025/2026

Grupo 03

Carlos Silva; Francisco Veloso; Nuno Soares
a106792@uminho.pt
a106882@uminho.pt
a107366@uminho.pt

Conteúdo

1	Introdução	3
1.1	Âmbito e decisões iniciais	3
1.2	Organização do relatório	3
2	Desenvolvimento	4
2.1	Arquitetura do compilador	4
2.2	Análise léxica	4
2.3	Análise sintática e gramática	4
2.4	Análise semântica	5
2.5	Geração de código	5
2.6	Funcionalidades cobertas	6
2.7	Dificuldades encontradas	6
3	Execução e Validação	8
3.1	Dependências e instalação	8
3.2	Utilização da linha de comandos	8
3.3	Validação na EWVM	8
3.4	Testes automáticos	8
4	Conclusões	9

1 Introdução

Este relatório documenta o desenvolvimento de um compilador para um *subset* da linguagem Fortran 77, realizado no âmbito da unidade curricular de Processamento de Linguagens do ano letivo 2025/2026. O compilador recebe um programa escrito em Fortran, valida-o a nível da léxica, sintática e semântica e, quando o programa é considerado válido, traduz-o para código da EWVM, a máquina virtual de *stack* disponibilizada na unidade curricular.

O trabalho foi escrito em Python e assenta na biblioteca `PLY`, tal como o enunciado nos pede. A fase léxica recorre ao módulo `ply.lex` e a fase sintática ao módulo `ply.yacc`. A tradução é feita diretamente para a máquina virtual, sem uma fase autónoma de representação intermédia. Consequentemente, a etapa de valorização correspondente à otimização de código decidimos não abordar. Em contrapartida, foi implementado o caso de valorização relativo à definição e chamada de funções (`FUNCTION`), o que permite compilar o exemplo mais exigente do enunciado, a conversão de um número entre bases numéricas.

1.1 Âmbito e decisões iniciais

Uma das primeiras decisões foi a escolha entre o formato de colunas fixas do Fortran 77 original e o formato livre (*free-form*) das versões mais recentes da linguagem. Optámos pelo formato livre, por ser mais simples de tokenizar e por estar mais próximo da forma como os exemplos do enunciado se encontram escritos. A única convenção herdada do formato clássico que mantivemos foi a obrigatoriedade de as *labels* de instrução surgirem no início da linha, uma vez que essa restrição simplifica de forma apreciável o reconhecimento léxico. Os identificadores são tratados de modo *case-insensitive* e normalizados para maiúsculas, à semelhança do que sucede no Fortran real.

O *subset* suportado cobre integralmente o núcleo obrigatório do enunciado a nível da declaração de tipos e variáveis, expressões aritméticas, lógicas e relacionais, estruturas de controlo de fluxo (`IF-THEN-ELSE`, ciclos `DO` com *label* e `GOTO`) e operações básicas de entrada e saída (`READ` e `PRINT`). A este núcleo acrescenta-se o suporte a *arrays* unidimensionais, à função intrínseca `MOD` e, como já referido, à definição e invocação de funções. O nosso compilador foi validado contra os cinco programas de exemplo do enunciado — `hello`, `fatorial`, `primo`, `somaarr` e `conversao` disponibilizados pelos professores.

1.2 Organização do relatório

O capítulo *Desenvolvimento* descreve a arquitetura do compilador e detalha cada uma das suas fases, apresenta a gramática utilizada e discute as principais dificuldades encontradas. O capítulo *Execução e Validação* explica como instalar e correr o compilador e que estratégia de testes foi adotada. O relatório encerra com as conclusões e o trabalho futuro.

2 Desenvolvimento

2.1 Arquitetura do compilador

O compilador está organizado como um pacote Python em `src/compilador` e segue a estrutura clássica de um compilador que é dividido em fases bem delimitadas. Cada fase tem um módulo próprio e comunica com a seguinte através de uma estrutura de dados explícita, o que torna o fluxo fácil de seguir e de testar isoladamente.

O módulo `lexer.py` contém o analisador léxico; o `parser.py` contém o analisador sintático e constrói a árvore de sintaxe abstrata (AST), cujos nós estão definidos em `ast.py`; o `semantics.py` valida a AST e anota-a com informação de tipos; o `codegen_vm.py` traduz a AST para instruções da EWVM. O módulo `compiler.py` encadeia estas fases numa *pipeline* única, e o `cli.py` oferece uma interface de linha de comandos que permite parar a compilação no fim de qualquer uma das fases. A passagem de informação entre fases é, portanto, sempre unidirecional: texto $\rightarrow$ *tokens* $\rightarrow$ AST $\rightarrow$ AST validada $\rightarrow$ código da máquina virtual. Caso uma fase detete um erro, lança uma exceção própria e a compilação termina, evitando que a fase seguinte trabalhe sobre dados inconsistentes.

2.2 Análise léxica

O analisador léxico, construído com `ply.lex`, converte o texto-fonte numa sequência de *tokens*. São reconhecidas as palavras reservadas da linguagem (PROGRAM, END, INTEGER, REAL, LOGICAL, IF, THEN, ELSE, ENDIF, DO, CONTINUE, GOTO, READ, PRINT, FUNCTION e RETURN), os identificadores, os literais inteiros, reais e de cadeia de caracteres, os literais lógicos `.TRUE.` e `.FALSE.`, e ainda os operadores aritméticos e os operadores relacionais e lógicos escritos na notação pontuada própria do Fortran (`.EQ.`, `.NE.`, `.LT.`, `.LE.`, `.GT.`, `.GE.`, `.AND.`, `.OR.` e `.NOT.`). Os comentários iniciados por `!` são descartados até ao fim da linha (porque são comentários e não necessários para a execução do programa).

Duas das características do analisador que construímos são que o fim de linha não é ignorado: o caractere de mudança de linha origina um *token* `NEWLINE`, porque o Fortran é uma linguagem orientada à linha e a gramática precisa de saber onde cada instrução termina. A segunda é a existência de um *token* dedicado para as *labels* de instrução, reconhecido por uma expressão regular ancorada ao início da linha. Isto distingue, sem ambiguidade, o número que rotula uma instrução de um literal inteiro qualquer. Sempre que se encontra um caractere inválido, o analisador lança uma exceção que indica a linha e a coluna do problema, informação que é aproveitada nas mensagens de erro apresentadas ao utilizador.

2.3 Análise sintática e gramática

O analisador sintático usa `ply.yacc`, que gera um *parser* LALR a partir das regras de produção. Cada regra constrói diretamente o nó da AST correspondente, de modo que, no final do *parsing*, existe já uma representação estruturada do programa. A unidade de compilação é um `PROGRAM` seguido de zero ou mais definições de `FUNCTION`. As regras centrais da gramática, em forma simplificada, são as seguintes:

```
1 compilation_unit : program function_defs
2 program          : PROGRAM IDENTIFIER NEWLINE program_items END
3 function_def     : type_spec FUNCTION IDENTIFIER LPAREN param_list_opt RPAREN
4                  : NEWLINE program_items END
5
6 program_items   : program_items program_item | empty
7 program_item    : declaration NEWLINE | statement | NEWLINE
8 statement       : label_opt simple_statement NEWLINE
9                  | label_opt if_statement
10                 | label_opt do_statement
11
12 declaration     : type_spec decl_list
13 decl_item       : IDENTIFIER | IDENTIFIER LPAREN INTEGER_LITERAL RPAREN
14
15 assignment      : designator ASSIGN expression
```

```

16 designator      : IDENTIFIER | IDENTIFIER LPAREN expression RPAREN
17
18 if_statement    : IF LPAREN expression RPAREN THEN NEWLINE statement_block
19               endif_tail
19 endif_tail      : ENDIF NEWLINE | ELSE NEWLINE statement_block ENDIF NEWLINE
20
21 do_statement    : DO INTEGER_LITERAL IDENTIFIER ASSIGN expression COMMA expression
22               do_step_opt NEWLINE do_block do_end
23 do_end          : LABEL CONTINUE NEWLINE
24
25 simple_statement : assignment | goto_statement | continue_statement
26               | return_statement | read_statement | print_statement

```

As expressões são descritas por uma única produção recursiva que cobre os operadores aritméticos, relacionais e lógicos. A ambiguidade natural que pode surgir através desta formulação que optámos é resolvida por uma tabela de precedências declarada no *parser*, que fixa a prioridade relativa dos operadores (do menos prioritário, `.OR.`, ao mais prioritário, `**` e o menos unário) e a respetiva associatividade:

```

1 expression : expression OR expression | expression AND expression
2           | expression EQ expression | expression NE expression
3           | expression LT expression | expression LE expression
4           | expression GT expression | expression GE expression
5           | expression PLUS expression | expression MINUS expression
6           | expression TIMES expression | expression DIVIDE expression
7           | expression POWER expression
8           | NOT expression | MINUS expression %prec UMINUS
9           | LPAREN expression RPAREN
10          | IDENTIFIER | IDENTIFIER LPAREN call_arg_list_opt RPAREN
11          | INTEGER_LITERAL | REAL_LITERAL | STRING_LITERAL | BOOL_LITERAL

```

Perante um erro sintático, o *parser* produz uma mensagem que localiza o *token* inesperado, distinguindo o caso de um fim de ficheiro prematuro.

2.4 Análise semântica

A análise semântica percorre a AST construída pelo *parser* e verifica tudo aquilo que a gramática, por si só, não consegue garantir. São validados o uso de variáveis previamente declaradas, a compatibilidade de tipos nas atribuições e nas expressões, o uso de *arrays* com exatamente um índice inteiro, a existência das *labels* referenciadas por `GOTO` e a correspondência entre a *label* de um ciclo `DO` e a *label* do `CONTINUE` que o fecha. No caso das funções, verifica-se que cada função contém pelo menos um `RETURN`, que o `RETURN` só ocorre dentro de funções, que o número e os tipos dos argumentos coincidem com a assinatura, e ainda que não existe recursão direta nem escrita sobre parâmetros formais.

Ao contrário das fases anteriores, a análise semântica não interrompe a verificação no primeiro problema: os erros vão sendo acumulados, cada um com a sua posição no código-fonte, e são todos reportados em conjunto no final. Esta opção torna a ferramenta bastante mais cómoda de usar, pois evita o ciclo de corrigir um erro de cada vez. Além de validar, esta fase desempenha um segundo papel: à medida que infere o tipo de cada expressão, anota esse tipo no próprio nó da AST. Essa informação fica assim disponível para a geração de código, que deixa de precisar de a recalculá-la.

2.5 Geração de código

O gerador de código traduz a AST validada para a sintaxe oficial da EWVM. As variáveis globais são colocadas em posições consecutivas da memória global, reservada de uma só vez com a instrução `PUSHN`, e acedidas por *offset*. Os *arrays* seguem o mesmo princípio, ocupando tantas posições quantas o seu tamanho. O endereço de um elemento `A(i)` é calculado a partir do endereço base somado ao índice, que é previamente normalizado para base zero (`i - 1`), já que o Fortran indexa a partir de um.

As estruturas de controlo são traduzidas com saltos condicionais e *labels* geradas internamente. O ciclo DO tem o cuidado adicional de suportar passos positivos e negativos, escolhendo em tempo de execução, conforme o sinal do passo, a comparação que decide a continuação do ciclo. As funções usam uma convenção de chamada simples em que os argumentos são empilhados, a chamada é feita com PUSHJ seguido de CALL, e dentro da função os parâmetros são acedidos em *offsets* negativos relativos ao *frame pointer*, enquanto o valor de retorno e as variáveis locais ocupam *offsets* não negativos. O valor de retorno é colocado na pilha e a função termina com RETURN.

Para cada operador binário, o gerador escolhe entre a instrução inteira e a instrução de vírgula flutuante consoante o tipo dos operandos. É neste caso aqui que a anotação de tipos produzida pela análise semântica que fizemos é aproveitada já que o gerador apenas consulta o tipo já registado em cada nó, não voltando a inferi-lo.

2.6 Funcionalidades cobertas

O compilador que fizemos suporta a totalidade do núcleo obrigatório do que nos foi pedido no enunciado e ainda a definição e chamada de funções. Ficaram de fora três aspetos. As SUBROUTINES não foram implementadas, tendo o nosso grupo priorizado o trabalho nas funções. Além disso, os *arrays* estão limitados a uma dimensão, não havendo suporte para *arrays* multidimensionais nem para sub-seções de *array*. Por fim, o operador de exponenciação ** é reconhecido pelo analisador léxico e pelo *parser*, mas não é traduzido pelo gerador de código que fizemos, por não existir uma instrução direta correspondente na versão da EWVM utilizada. Se houver um programa que o use é aceite até à análise semântica e é rejeitado depois, de forma explícita, na geração de código.

2.7 Dificuldades encontradas

A dificuldade mais interessante do ponto de vista de processamento de linguagens foi a ambiguidade da forma sintática ID(...) que o nosso grupo notou. Em Fortran, o acesso a um elemento de um *array* e a chamada de uma função escrevem-se exatamente da mesma maneira, e a gramática, sozinha, não dispõe de informação para os distinguir. Esta distinção depende do identificador ter sido declarado como *array* ou de existir uma função com aquele nome. A solução a que chegamos passou por adiar esta decisão. Ou seja, nas expressões, o *parser* produz sempre um nó genérico de chamada, e é a análise semântica que, consultando a tabela de símbolos e as assinaturas de funções recolhidas previamente, decide se se trata de um acesso indexado ou de uma invocação.

Uma segunda dificuldade que sentimos foi que, ao contrário de linguagens em que o fim de linha é irrelevante, aqui ele delimita instruções, pelo que foi preciso decidir tratá-lo como um *token* significativo e refletir essa opção em toda a gramática. No mesmo registo, o reconhecimento das *labels* de instrução obrigou a uma expressão regular ancorada ao início da linha, para que um número usado como rótulo não fosse confundido com um literal inteiro presente numa expressão.

Desta mesma maneira, a coerência das *labels* dos ciclos DO exigiu igualmente atenção. O enunciado pedia-nos explicitamente a validação de que a *label* do DO corresponde à do CONTINUE que o termina, e essa verificação teve de ser articulada com a resolução das *labels* usadas por GOTO, de modo a garantir que todas as referências apontam para rótulos efetivamente definidos.

De seguida, a geração de código exigiu um estudo cuidadoso do conjunto de instruções da EWVM e, em particular, da convenção de chamada de funções. Foi necessário fixar onde ficam os parâmetros e as variáveis locais relativamente ao *frame pointer*, como é transportado o valor de retorno e como se mapeia a memória dos *arrays* sem recorrer a alocação dinâmica. A validação foi feita executando o código gerado no interpretador oficial da EWVM (que os professores partilharam) e comparando os resultados com o comportamento esperado.

Por último, uma decisão de arquitetura revelou-se mais importante do que nos parecia à partida, ao nível de onde a informação de tipos é calculada. Numa primeira versão, o gerador de código voltava a inferir os tipos das expressões a partir da AST, repetindo lógica que já existia na análise semântica. Essa duplicação não só era desnecessária como abria a porta a divergências entre as duas fases (bastava as duas inferências discordarem para o gerador escolher, por exemplo, uma comparação inteira onde era precisa uma comparação de vírgula flutuante). A solução foi centralizar a inferência, a nossa análise semântica passou a anotar o tipo

de cada expressão no respetivo nó da AST, e o gerador de código limitou-se a consultar essa anotação. Desta maneira, a escolha das instruções da máquina virtual ficou garantidamente coerente com a verificação de tipos.

3 Execução e Validação

3.1 Dependências e instalação

A única dependência externa do compilador é a biblioteca PLY. Num ambiente com Python 3 instalado, basta executar `pip install ply` para a obter. Todo o restante código é Python puro e não requer qualquer compilação ou configuração adicional.

3.2 Utilização da linha de comandos

O compilador é invocado como um módulo Python. Como o código está em `src/compilador`, a diretoria `src` tem de constar do caminho de procura do Python, o que se consegue colocando `PYTHONPATH=src` antes do comando. A ferramenta aceita um ficheiro de entrada, um modo de operação e, opcionalmente, um ficheiro de saída.

O modo é escolhido com a opção `--mode` e determina até onde a compilação avança. O modo `lex` apresenta apenas a lista de *tokens*. O modo `parse` mostra a árvore de sintaxe abstrata; o modo `sem` corre a análise semântica e indica os símbolos reconhecidos. O modo `vm` executa a *pipeline* completa e produz o código da máquina virtual. Estes modos intermédios revelaram-se valiosos durante o desenvolvimento, por permitirem inspecionar o resultado de cada fase de forma isolada (melhor para debug). A geração de código para um dos exemplos faz-se da seguinte forma:

```
1 mkdir -p output
2 PYTHONPATH=src python3 -m compilador --mode vm Testes/text/hello.txt -o output/
   hello.vm
```

Quando a opção `-o` é indicada, o código gerado é gravado nesse ficheiro. Caso contrário, é apenas escrito no terminal.

3.3 Validação na EWVM

A validação funcional foi feita no interpretador oficial da EWVM, disponível em <https://ewvm.epl.di.uminho.pt/>. O procedimento consistiu em gerar o ficheiro `.vm` correspondente a cada exemplo, copiar o conteúdo para o editor da máquina virtual e executá-lo, fornecendo *input* sempre que o programa o pediam, como acontece nos exemplos `primo` e `conversao`, que leem números do utilizador. Os cinco programas do enunciado foram compilados e os respetivos ficheiros de código da máquina virtual estão incluídos na entrega, na diretoria `output`.

3.4 Testes automáticos

Para complementar a validação manual na EWVM e proteger o projeto contra regressões, desenvolvemos alguns testes automáticos, escrita com o módulo `unittest` e organizada em `Testes/py`. Os testes estão divididos por fase do compilador: `test_lexer.py` verifica a tokenização, o tratamento *case-insensitive* e a deteção de caracteres ilegais; `test_parser_semantics.py` confirma o *parsing* de programas válidos e a deteção de erros sintáticos e semânticos típicos, como o uso de variáveis não declaradas; `test_codegen_pipeline.py` valida a compilação ponta-a-ponta, incluindo a compilação de todos os exemplos do enunciado;

A suite de testes é executada a partir da raiz do projeto com o comando seguinte:

```
1 python3 -m unittest discover -s Testes/py -v
```


4 Conclusões

De forma geral, o trabalho cumpre os objetivos definidos no enunciado. Foi construído um compilador completo para um *subset* de Fortran 77, com análise léxica, sintática e semântica, capaz de traduzir programas para código executável na EWVM. Para além desse núcleo, foi também coberto o caso de valorização relativo às funções, incluindo a validação de parâmetros e a geração de chamadas com **CALL** e **RETURN**, o que permite compilar e executar o exemplo de conversão de bases. Os cinco programas de exemplo do enunciado são compilados com sucesso e a sua execução é confirmada no interpretador oficial da máquina virtual.

Mais do que o resultado final, o desenvolvimento foi proveitoso pela forma como obrigou a aplicar, num caso concreto, os conceitos da unidade curricular a nível da separação clara entre fases, a construção de uma árvore de sintaxe abstrata como representação intermédia, a resolução de ambiguidades que a gramática não consegue decidir sozinha e o cuidado em manter uma única fonte de verdade para a informação de tipos.

Ainda assim, subsistem algumas limitações, que delimitam com clareza o trabalho futuro. A mais relevante que achamos é a ausência de **SUBROUTINES**, cuja implementação seria a extensão natural do suporte já existente para funções, caso continuássemos a desenvolver este projeto. Além disso, o suporte a *arrays* multidimensionais e a tradução do operador ******, que exigiria expandi-lo para multiplicações sucessivas ou recorrer a uma outra rotina auxiliar. Finalmente, a etapa de otimização de código, deixada de fora por se ter optado pela tradução direta para a máquina virtual, seria o passo seguinte caso se pretendesse introduzir uma representação intermédia autónoma e melhorar a qualidade do código gerado.

Referências

1. David Beazley, *PLY (Python Lex-Yacc)*, <https://www.dabeaz.com/ply/>.
2. EPL/DI/UMinho, *EWVM online*, <https://ewvm.epl.di.uminho.pt/>.
3. Unidade Curricular de Processamento de Linguagens, *Construção de um Compilador para Fortran 77 Standard – Projeto de Processamento de Linguagens 2026*, Universidade do Minho, 2026.
4. American National Standards Institute, *American National Standard Programming Language FORTRAN, ANSI X3.9-1978*.